

# Algorithms for Data Science

## Cheatsheet for Exercises 5

In a dictionary, each data entry that we store consists of two components: a *key* (often a number) and a value (any data type). The entries are therefore called *key-value pairs*.

*Example: key = student's ID, value = student's name and address.*

**Invariant (Dictionary):** *Each key appears at most once in a dictionary.*

Note that there are no such restrictions on the values. The same value can appear with different keys.

The method `Add(k, v)` first checks whether the dictionary contains an entry with the key equal to  $k$ . If it contains, the old value is replaced by the new value  $v$ . If it does not contain such an entry, a new entry is stored: with the key  $k$  and the value  $v$ .

The dictionary **hash table** is an array together with a hash function that, given a key, computes the position in the array that is responsible for this key.

Hash function have often the following form:  $h(k) = k \bmod p$ , where  $p$  is the size of the array and where  $\bmod$  is the modulo operator.

**Definition of Modulo:** If  $k \geq 0$  and  $p > 0$  are integers, then “ $k$  modulo  $p$ ” (alternative writing:  $k \bmod p$  and  $k \% p$ ) is

- the remainder of the division  $k/p$ , formally:
- $k - p \cdot \lfloor k/p \rfloor$ .

A simple algorithm for computing  $k \bmod p$ :

---

```
Modulo( $k, p$ )
  while  $k \geq p$  do
     $k = k - p$ 
  return  $k$ 
```

---

The dictionary **AVL tree** is a **balanced binary search tree**.

Definitions:

- A tree is *binary* if each node has at most two children (called left and right).
- A binary tree is a binary search tree (BST) if each node has a key and satisfies the BST invariant (see below).
- A tree is balanced if its height is  $O(\log n)$  where  $n$  is the number of nodes.
- The *height of a tree* is the height of its root node.
- The *height of a node  $v$*  is the length of a longest path starting in  $v$  and going always to the bottom until it reaches a node that has no children. The length of the path is measured by the number of nodes that we visit including  $v$ . Thus, if  $v$  has not children, then its height is 1.
- An AVL tree is a balanced BST where each node satisfies the AVL tree invariant (see below).

In the exam, short tests or exercise sessions, when you are asked for an algorithm for a binary search tree, you can make the following assumptions:

- We have the variable type `node` that corresponds to a node in a binary search tree.
- In most algorithms, you will start with the node called *root* that corresponds to the root of the tree.
- If the tree is empty, then `root==NONE` (meaning that the root is not set up yet).
- If  $v$  is a node (and `v  $\neq$  NONE`), then it has the following attributes:
  - `v.key` is the key stored in  $v$ ,
  - `v.value` is the value stored in  $v$ ,
  - `v.left` is a pointer to the left child of  $v$  (set to `NONE` if it does not have a left child),
  - `v.right` is a pointer to the right child of  $v$  (set to `NONE` if it does not have a right child), and
  - `v.parent` is a pointer to the parent of  $v$ . If  $v$  is the root of the tree, then its parent is set to `NONE`.

Here an example of a sequence of commands creating a node with key 5 and value 123456 that has a right child whose key is 6 and whose value is unspecified:

```
node v = new node
v.key = 5
v.value = 123456
v.left = None
w = new node
w.key = 6
v.right = w
w.parent = v
```

**Binary Search Tree (BST) invariant:** *For every node  $v$ , all keys in its left subtree are strictly less than the key of  $v$ , and all keys in its right subtree are strictly greater than the key of  $v$ .*

**AVL tree invariant:** *For every node  $v$  with at least one child: If  $v$  has only one child, then its child has height 1. If  $v$  has two children, then the heights of the children of  $v$  differ by at most 1.*

A node  $v$  without a left or a right child has `v.left==NONE` or `v.right==NONE`, respectively. If we define “NONE” to be a virtual/dummy node of height 0, then we can simplify the AVL tree invariant as follows.

**AVL tree invariant (simplified):** *For every node  $v$ , the heights of the children of  $v$  differ by at most 1.*

A **Heap** is not a dictionary and **not** a binary search tree! However, it is a complete binary tree. *Complete* means that the tree looks identical to tree obtained by filling the levels (nodes of the same height) one by one from top to bottom, and each level from left to right. A heap can store just values, or value-priority pairs. In the former case, the heap invariant is with respect to the values, in the latter case, with respect to the priorities.

**Heap invariant:** *For every node  $v$ , its priority is not smaller than the priority of its children!*

Thus, a heap may contain multiple value-priority pairs with the same priority!